

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	M Jeevitha	Reg. No.:	192424257
Section / Batch:	Slot c -2024	Date:	25-8-2026

Code of Conduct

*I **M Jeevitha(192424257)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Jeevitha

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach

Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	4	4	3	3	3	17	92
2	4	4	4	4	4	20	
3	4	4	4	3	3	18	
4	4	4	3	4	3	18	
5	4	4	4	4	3	19	

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Reference Resolution Using Multiple Constraints

Given discourse:

“Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it.”

Algorithm

Step	1:	Identify	all	entities	and	pronouns.			
Step	2:	Generate	possible	antecedents	for	each	pronoun.		
Step	3:	Check	gender	and	number	agreement.			
Step	4:	Check	recency	and	grammatical	role.			
Step	5:	Check	semantic	compatibility	with	the	action.		
Step	6:	Select	the	antecedent	with	the	highest	overall	score.

Step 7: Replace pronouns with their resolved entities.

Scoring idea

A simple scoring method can be:

$$\text{Score} = \text{Gender} + \text{Number} + \text{Recency} + \text{Semantic Compatibility}$$

The antecedent with the highest score is selected.

Step 1: Entities and referring expressions

Entities:

- Meena
- Kavya
- laptop
- project

Referring expressions:

- she
- one
- She
- her
- it

Step 2–4: Possible antecedents and resolution

Referring Expression	Possible Antecedents	Resolution Reason	
she	Meena, Kavya	Kavya	Both are female, but Kavya is more recent and “needed one for her project” fits the recipient.
one	laptop	laptop	“One” refers to an object of the same type as laptop.
She	Meena, Kavya	Kavya	Kavya is the most recent female entity and fits the following

Referring Expression	Possible Antecedents	Resolution Reason
		action.
her	Meena, Kavya	Meena “She thanked her” means Kavya thanked Meena; grammatical role and discourse meaning support this.
it	laptop	laptop “Using it” semantically refers to the laptop.

Important ambiguity

The first “**she**” could grammatically refer to either Meena or Kavya because both are female and singular.

However:

“Meena gave Kavya a laptop because **she needed one** for her project.”

The preferred interpretation is:

she → **Kavya**

because Kavya is the more recent entity and the person receiving the laptop is the most natural person to need it for a project.

Final resolved discourse

Meena gave Kavya a laptop because Kavya needed the laptop for Kavya's project. Kavya thanked Meena and started using the laptop.

How does the algorithm avoid incorrect resolution?

When multiple antecedents have the same gender and number, the algorithm does **not depend only on gender and number**.

It additionally checks:

- **Recency** → prefers recently mentioned entities.
- **Grammatical role** → considers subject/object roles.
- **Semantic compatibility** → checks whether the entity makes sense with the verb.
- **Discourse coherence** → checks whether the interpretation makes the overall discourse logical.

Therefore, multiple constraints together reduce incorrect resolutions.

\

2. Algorithm for Entity Chains and Discourse Coherence

Given text:

“Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application.”

Algorithm

Step	1:	Divide	the	text	into	sentences.
Step	2:	Identify	nouns	and	referring	expressions.
Step	3:	Compare	expressions	using	semantic similarity	and reference rules.
Step	4:	Link	expressions	referring	to	the same entity.
Step	5:	Construct	entity	chains.		

Step 6: Measure how frequently entities continue across sentences.

Step 7: Determine discourse coherence.

Step 1: Entities and referring expressions

Expression Entity

Anjali	Anjali
laptop	Laptop
The laptop	Laptop
She	Anjali
it	Laptop
Anjali	Anjali
the computer	Laptop

Step 2–3: Entity chains

Entity Chain 1 — Anjali

Anjali → She → Anjali

Appears in sentences:

S1 → S3 → S4

Entity Chain 2 — Laptop

laptop → The laptop → it → the computer

Appears in:

S1 → S2 → S3 → S4

Step 4: Continuity

Sentence Anjali Laptop

S1	✓	✓
S2	—	✓
S3	✓	✓
S4	✓	✓

The laptop is maintained throughout almost every sentence, while Anjali reappears after being introduced.

Continuity estimate

There are strong links between consecutive sentences:

- S1 → S2: **Laptop continues**
- S2 → S3: **Laptop continues + Anjali returns**
- S3 → S4: **Anjali + Laptop continue**

Therefore, entity continuity is **high**.

Step 5: Is the discourse coherent?

Yes, the discourse is coherent.

The text maintains two main entities:

Anjali → She → Anjali

and

Laptop → The laptop → it → the computer

Every sentence logically develops the previous information.

Why?

The discourse follows a logical progression:

Anjali buys laptop → laptop has processor → Anjali installs Python → Anjali uses laptop for ML application.

What happens if an entity chain is broken?

If we change the final sentence to:

“Later, **Rahul** used the computer to develop a machine learning application.”

A new entity, Rahul, suddenly appears without introduction.

This breaks the **Anjali entity chain** and may reduce coherence because the reader may wonder:

“Who is Rahul and why is he involved?”

Exam answer:

Breaking an entity chain can reduce discourse coherence because the connection between sentences becomes weak. Readers may have difficulty identifying who or what the discourse is about.

3. Algorithm for Identifying Discourse Relations

Given discourse:

“The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again.”

Algorithm

- Step 1:** Divide the discourse into discourse units.
- Step 2:** Detect explicit discourse markers.
- Step 3:** Analyze the meaning of consecutive units.
- Step 4:** Determine the relation: Cause–Effect, Sequence, Problem–Solution, or Elaboration.
- Step 5:** Build a discourse structure.

Step 1: Discourse units

DU1:

The server crashed unexpectedly.

DU2:

As a result, users could not access the application.

DU3:

The technical team restarted the server.

DU4:

After that, the system became available again.

Step 2: Discourse markers

Marker	Meaning
---------------	----------------

As a result	Cause–Effect
--------------------	--------------

After that	Sequence
-------------------	----------

Step 3–4: Relations

Units	Relation	Explanation
--------------	-----------------	--------------------

DU1 → DU2	Cause–Effect	Server crash caused users to lose access.
-----------	---------------------	---

DU2 → DU3	Problem–Solution	Access problem led to the technical team restarting the server.
-----------	-------------------------	---

DU3 → DU4	Sequence	Restarting happened before the system became available.
-----------	-----------------	---

DU4	Result/Resolution	System becomes available again, resolving the problem.
-----	--------------------------	--

Discourse structure representation

You can write:

DU1: Server crashed

↓

CAUSE–EFFECT

↓

DU2: Users could not access application

↓

PROBLEM–SOLUTION

↓

DU3: Team restarted server

↓

SEQUENCE

↓

DU4: System became available

Why is it coherent?

The events follow a clear logical order:

Crash → Access problem → Restart → Recovery

The discourse markers “**As a result**” and “**After that**” explicitly indicate the relationships between events.

Exam conclusion:

The identified discourse relations make the text coherent because each sentence is logically connected to the

previous sentence through cause, problem, solution, and sequence relationships.

4. Algorithm for Dialogue Act Classification

Conversation

User: “Hello, I need help with my assignment.”

Agent: “Sure, what problem are you facing?”

User: “I do not understand machine learning.”

Agent: “I can explain the basic concepts to you.”

Algorithm

Step 1: Preprocess the utterance.
Step 2: Extract linguistic features such as keywords, question words, and sentence structure.
Step 3: Determine the speaker's communicative intention.
Step 4: Match the intention with a dialogue-act category.
Step 5: Generate the dialogue-act sequence.

Step 1–4: Classification

Speaker Utterance	Important Features	Dialogue Act
User “Hello, I need help with my assignment.”	“Hello”, “need help”	Greeting + Request
Agent “Sure, what problem are you facing?”	Question form	Question
User “I do not understand machine learning.”	Provides information	Inform
Agent “I can explain the basic concepts to you.”	Offers assistance	Offer

Dialogue-act sequence

Greeting → Request → Question → Inform → Offer

Or:

User: Greeting + Request

↓

Agent: Question

↓

User: Inform

↓

Agent: Offer

Why is dialogue-act recognition useful?

Dialogue-act recognition helps a conversational agent understand **what the user is trying to do**.

For example:

- **Greeting** → respond with a greeting.
- **Request** → provide help.

- **Question** → provide an answer.
- **Inform** → process the information.
- **Offer** → accept or respond to the offered help.

Therefore, it helps the agent select an **appropriate next response** and maintain a natural conversation.

5. Algorithm for Dialogue State Tracking

Conversation

User: “I want to book a flight.”

Agent: “Where would you like to travel?”

User: “I want to go to Delhi.”

Agent: “From which city?”

User: “From Chennai.”

Required slots

1. Departure City
2. Destination City
3. Travel Date
4. Number of Passengers

Algorithm

Step 1: Initialize all slots as empty.

Step 2: Read each user utterance.

Step 3: Extract entities and values.

Step 4: Match each value to the correct slot.

Step 5: Update the dialogue state.

Step 6: Identify missing slots.

Step 7: Ask a question for the next missing slot.

Step 1: Initial state

Slot	Value
Departure City	Unknown
Destination City	Unknown
Travel Date	Unknown
Number of Passengers	Unknown

Step 2: User says

“I want to book a flight.”

No slot value is provided.

State remains:

Slot	Value
Departure City	Unknown
Destination City	Unknown
Travel Date	Unknown
Number of Passengers	Unknown

Agent asks:

“Where would you like to travel?”

Step 3: User says

“I want to go to Delhi.”

The system identifies:

Delhi → Destination City

Updated state:

Slot	Value
Departure City	Unknown
Destination City	Delhi
Travel Date	Unknown
Number of Passengers	Unknown

Agent asks:

“From which city?”

Step 4: User says

“From Chennai.”

The system identifies:

Chennai → Departure City

Updated state:

Slot	Value
Departure City	Chennai
Destination City	Delhi
Travel Date	Unknown
Number of Passengers	Unknown

Step 5: Identify missing information

Still missing:

- **✗** Travel Date
- **✗** Number of Passengers

The next appropriate question could be:

“What date would you like to travel?”

After obtaining the date, the agent can ask:

“How many passengers will be travelling?”

Final dialogue state

Destination City = Delhi

Departure City = Chennai

Travel Date = Unknown

Number of Passengers = Unknown

How Dialogue State Tracking improves coherence

Dialogue State Tracking stores information from previous turns so the agent does not repeatedly ask for information that the user has already provided.

For example, after the user says **“From Chennai”**, the agent remembers:

Departure City = Chennai

and can move to the next missing slot instead of asking:

“Which city are you departing from?”

again.